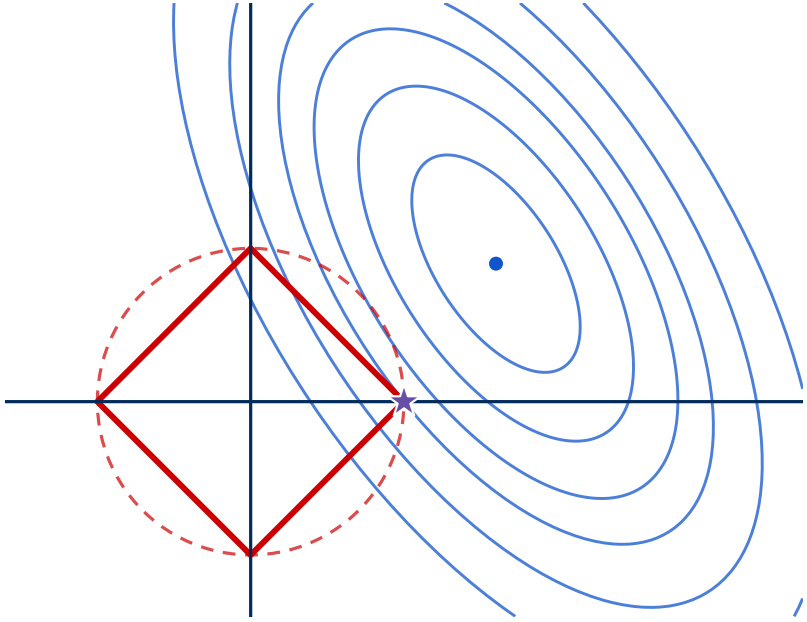


UNIT 02 · CHAPTER 3

Optimization and Regularization

CSE 30124 · Introduction to Artificial Intelligence
William Theisen · University of Notre Dame

Where we left off

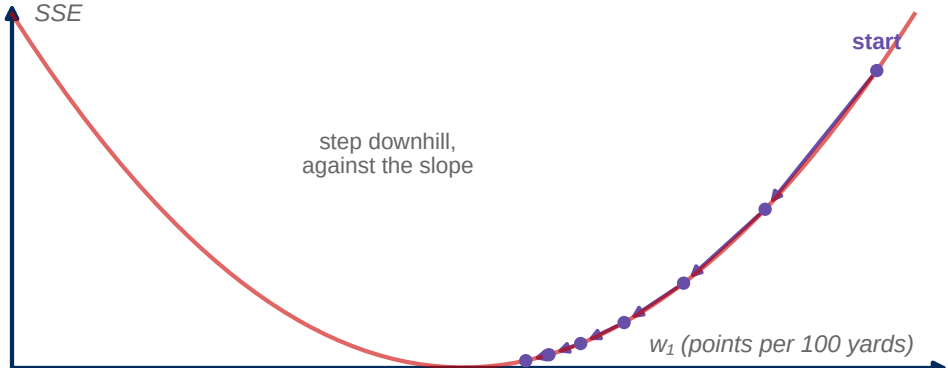
Last chapter we fit a line to Notre Dame's football games by writing down a loss, the SSE, and solving for where its gradient is zero. That gave us ordinary least squares, $\mathbf{w}^T = (X^T X)^{-1} X^T \mathbf{y}$, and a line worth about 9 points per 100 yards.

Unfortunately, OLS left us with **two problems**. Inverting $X^T X$ gets very expensive as the number of features grows, and when one feature can be built from the others (total yards is just pass yards plus rush yards) the inverse doesn't exist at all. On top of that, a model with enough features overfits, and its training error looks great right up until it meets a new game.

The iterative method takes care of the cost. Regularization takes care of the rest.

An iterative method

Instead of jumping straight to the bottom of the bowl, we can **walk downhill** a little at a time.



[3.1] one step of gradient descent

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial L}{\partial \mathbf{w}}$$

Here $\mathbf{w}$ is the whole list of weights, $\partial L / \partial \mathbf{w}$ is the gradient from the last chapter, and α is the **learning rate**, how big a step we take each time. Walking downhill fixes the cost problem, there's no inverse anywhere. Unfortunately, it doesn't fix the second problem. If $X^T X$ has no inverse the bottom of the bowl is a flat valley, and gradient descent will happily stop wherever in the valley it lands. We'll fix that one with regularization. (Gradient descent comes back properly with logistic regression, so this is just a preview.)

In code: walking downhill

Gradient descent is only a few lines more. Start anywhere, compute the gradient, take a small step against it, repeat:

```python
import numpy as np

x = np.array([2, 3, 4, 5, 6])
y = np.array([14, 20, 31, 40, 49])
X = np.column_stack([np.ones(len(x)), x])

w = np.zeros(2)      # start anywhere
alpha = 0.005        # learning rate
for step in range(5000):
    grad = -2 * X.T @ (y - X @ w)   # gradient of SSE
    w = w - alpha * grad            # step downhill
print(w.round(2))
```

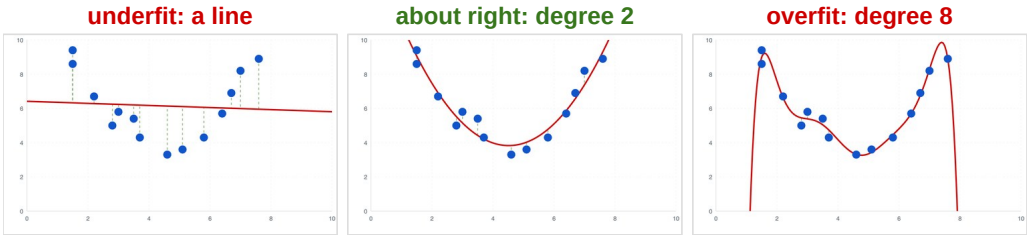
OUTPUT

```
[-5.2  9. ]
```

Same answer as the inverse, just found the slow way. Now try `alpha = 0.02`. Every number turns into `nan` within a few hundred steps, because a step that big jumps right over the bottom of the bowl and lands higher up the other side, and each step after that makes it worse. **Picking the learning rate is most of the work** in getting gradient descent to behave.

Overfitting

Unfortunately, **Linear Regressions** (a form of MLE), are **prone to overfitting**.



Live: open the degree-8 fit in the visualizer, then remove terms or drag a point and watch the curve swing.

This is where the last chapter left off. A model with enough features will happily fit the noise, and the training error can't warn us. The rest of this chapter is about the fix.

Regularization: L_1 and L_2

To help **prevent overfitting** when using **linear regressions**, we can perform a technique called **regularization**.

The goal of **regularization** is to **encourage models to learn the broader patterns** within the data rather than memorizing it.

L_2 regularization: Ridge

L2 regularization (called **ridge regularization**) **encourages small (but nonzero) weights**.

L_1 regularization: LASSO

L1 regularization (also called **LASSO**) leads to weights of exactly zero and **sparse models**.

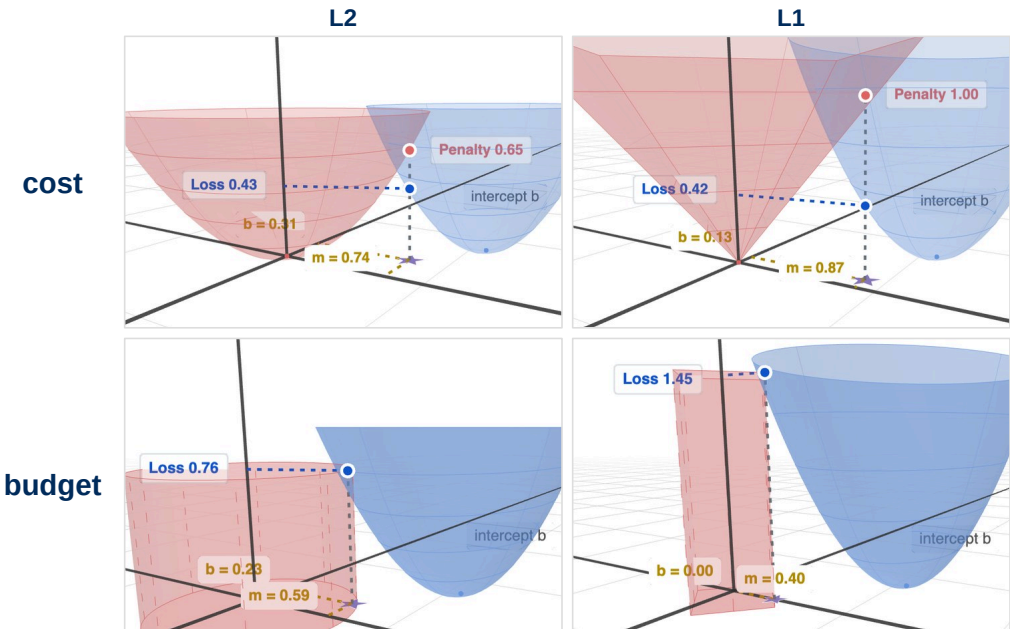
Why small weights?

Why would big weights mean overfitting? Look back at the degree-8 curve through the U-shaped points. To swing through every point it needs weights in the hundreds and thousands (the biggest is about 1,023), each one mostly cancelling out the one next to it. The straight line's biggest weight is about 6 and the degree-2 curve's is about 16. A model that needs **enormous, finely balanced weights** is almost always fitting the noise.

Formalizations of regularization

There are two ways to formalize regularization:

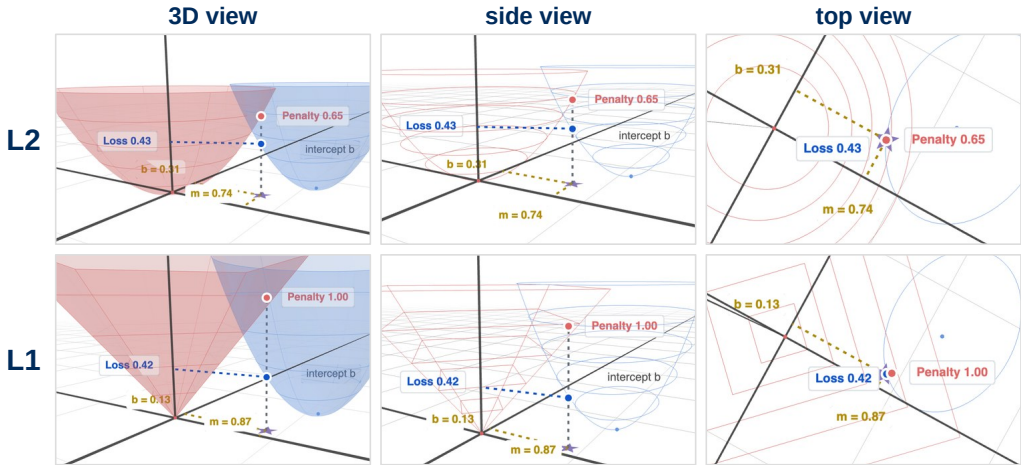
- As a cost (penalty) placed on the **weights**
- As a budget (constraint) for the **weights**



Live: open the regularization visualizer. Every picture in this section is a snapshot of it, so you can change the penalty, λ or t yourself.

Regularization as a cost

When we formalize **regularization** as a **cost**, we modify the **loss function** such that increasing the weights incurs a "cost".



We use an argument called λ to determine how heavily the **cost** is weighed. A larger λ increases the **cost**.

L2 as a cost

$$[3.2] \quad L(w) = SSE(w) + \lambda \sum_j w_j^2$$

L1 as a cost

$$[3.3] \quad L(w) = SSE(w) + \lambda \sum_j |w_j|$$

Ridge fixes the inverse

Remember the possible issue from the last chapter, $X^T X$ not having an inverse? Ridge fixes it almost by accident. Take the gradient of the ridge loss, set it to zero like we did for OLS, and the only change is a λ added down the diagonal:

[3.4] ridge, solved in one step

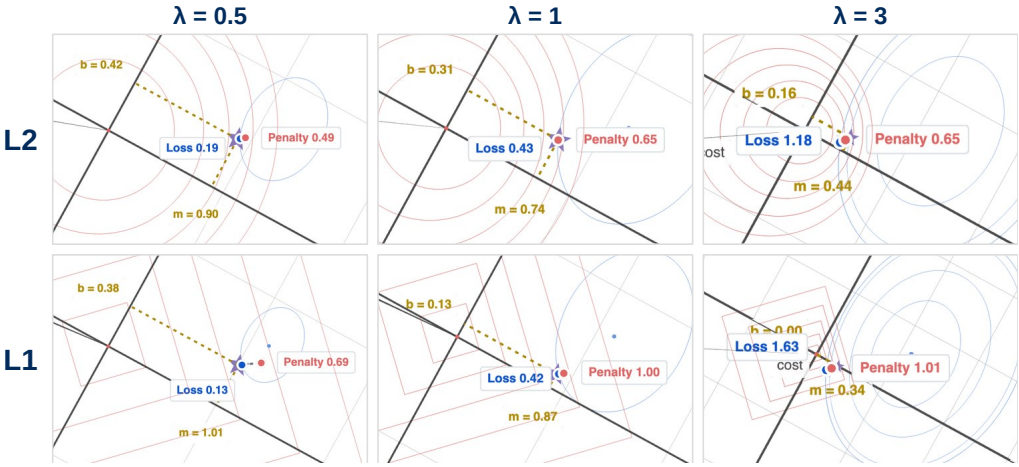
$$w^T = (X^T X + \lambda I)^{-1} X^T y$$

Adding any positive λ to the diagonal **guarantees the inverse exists**, so the flat valley turns back into a bowl with one bottom. In practice we usually leave the intercept w_0 out of the penalty (there's no reason to push the average score toward 0), so the top-left entry of I becomes a 0.

Back to the pass, rush and total yards. Plain OLS gave us weights of −64, +72 and +74. Ridge with $\lambda = 1$ gives **+5.9** on total yards, **+2.0** on pass yards and **+3.9** on rush yards. The predictions barely change (100 passing yards is still worth about 8 points and 100 rushing yards about 10), but now every weight is positive and none of them are absurd.

Regularization as a cost, increasing λ

A larger λ increases the **cost** of our weights, typically resulting in the optimum being pulled closer to the origin.



Regularization as a budget

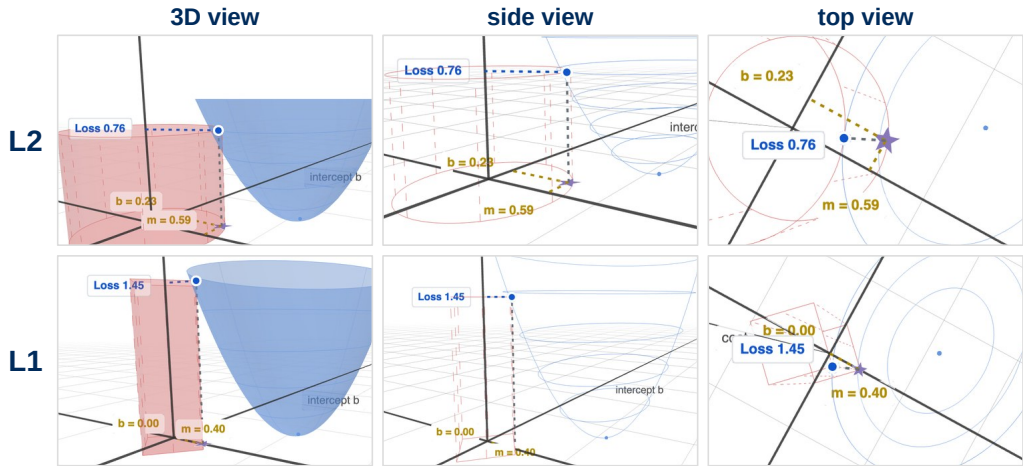
When we formalize **regularization** as a **budget**, we constrain our **loss function** such that the weights chosen must remain within "budget".

L2 as a budget

$$\min_{w} \text{SSE}(w) \quad \text{subject to} \quad \sum_{j} w_j^2 \leq t$$

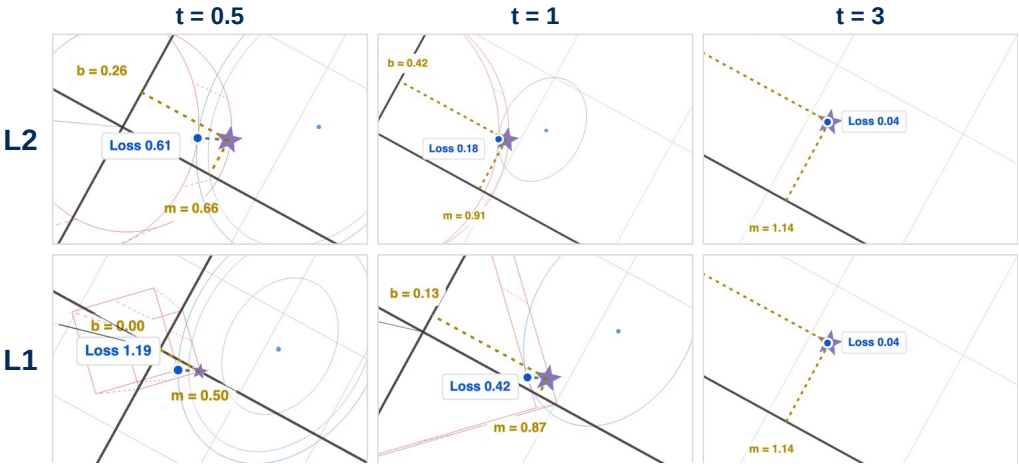
L1 as a budget

$$\min_{w} \text{SSE}(w) \quad \text{subject to} \quad \sum_{j} |w_j| \leq t$$



Regularization as a budget, increasing t

A larger t increases the **budget** for our weights, typically resulting in the optimum being closer to the unconstrained optimum.



Corners and budgets

Look at the L1 row of the cost pictures. At λ = 3 the star lands exactly on one of the axes, so one of the two weights is exactly 0. That's what "sparse" means. The budget pictures show why it happens: the L1 budget is a diamond, and the loss bowl almost always touches a diamond first at one of its corners, which sit right on the axes. The L2 budget is a circle with no corners, so its star gets pulled in close to an axis but never quite lands on it.

The cost and the budget aren't really two different ideas, they're the same problem written two ways. For every budget t there's some λ that puts the star in exactly the same spot (showing it takes Lagrange multipliers, which we'll see again with SVMs). **A big λ is a small budget**, and a small λ is a big one.

LASSO picks the features

A weight of exactly 0 means that feature is out of the model entirely, so L1 is doing **feature selection** for us for free. Let's give it something to select. Here are seven things from Notre Dame's box score: total yards, first downs, turnovers, penalties, third-down conversions, time of possession and number of plays.

Plain OLS uses all seven and explains 64% of the variation in points. LASSO with a moderate λ keeps exactly two of them, **total yards** (about 7.9 points per 100 yards) and **turnovers** (about −1.9 points each), and still explains 61%. The other five weights are exactly 0. Most of what a box score tells you about points, it turns out, is how far you moved the ball and how often you gave it away.

One practical detail: before using LASSO we rescale every feature to the same spread. Otherwise a feature measured in big units (yards) and one measured in small units (turnovers) get penalized very differently for the same amount of usefulness.

In code: ridge

sklearn has both. One annoying detail: sklearn calls λ `alpha`, which has nothing to do with the learning rate from gradient descent.

```
import pandas as pd
from sklearn.linear_model import LinearRegression, Ridge

games = pd.read_csv("nd_football.csv")
cols = ["nd_total_yards", "nd_pass_yards",
        "nd_rush_yards"]
X = games[cols] / 100
y = games["nd_points"]

ols = LinearRegression().fit(X, y)
ridge = Ridge(alpha=1).fit(X, y)   # alpha is λ here
print(ols.coef_.round(1))
print(ridge.coef_.round(1))
```

OUTPUT

```
[-64.4  72.4  74.2]
[5.9 2.  3.9]
```

Same predictions as OLS, sensible weights, exactly as on the ridge page.

In code: LASSO

LASSO needs one extra step first, rescaling every feature to the same spread:

```
import pandas as pd
from sklearn.linear_model import Lasso
from sklearn.preprocessing import StandardScaler

games = pd.read_csv("nd_football.csv")
stats = ["nd_total_yards", "nd_first_downs",
         "nd_turnovers", "nd_penalties",
         "nd_third_down_conv", "nd_possession_min",
         "nd_plays"]
X = StandardScaler().fit_transform(games[stats])
y = games["nd_points"]

lasso = Lasso(alpha=1).fit(X, y)
for name, w in zip(stats, lasso.coef_):
    print(f"{name:20s} {w + 0:6.2f}")
```

OUTPUT

```
nd_total_yards         8.16
nd_first_downs         0.00
nd_turnovers          -2.29
nd_penalties           0.00
nd_third_down_conv     0.00
nd_possession_min      0.00
nd_plays               0.00
```

These LASSO weights are per standard deviation (that's what `StandardScaler` did), so compare them to each other rather than to the 9 points per 100 yards from before.

MLE and MAP side by side

In the last chapter we said least squares is maximum likelihood in disguise. Now that we have regularization we can finish that thought, because MAP comes out of the same setup with one extra term.

The setup is an assumption about the noise. Every game's score is the line plus some noise, and the noise is Gaussian with spread σ: $y_i = \hat{y}_i + \varepsilon_i$ with $\varepsilon_i \sim N(0, \sigma^2)$. MLE picks the weights that make the scores we saw most likely, and taking the negative log turns that product of bell curves into a sum of squares:

[3.5]

$$\hat{w}_{MLE} = \arg\max_{w} \; p(y \mid w)$$

$$\downarrow \text{ take } -\log$$

$$= \arg\min_{w} \; \frac{SSE(w)}{2\sigma^2} = \arg\min_{w} \; SSE(w)$$

Adding a prior

MAP asks for the most likely weights given the data, which by Bayes' rule means multiplying in a prior $p(w)$. If we believe each weight is probably near 0, a Gaussian prior $w_j \sim N(0, \tau^2)$, its negative log is one more sum of squares:

$$\hat{w}_{MAP} = \arg\max_w \; p(y \mid w)\, p(w)$$

↓ take $-\log$

$$= \arg\min_w \; \frac{SSE(w)}{2\sigma^2} + \frac{1}{2\tau^2} \sum_j w_j^2$$

↓ multiply by $2\sigma^2$

$$= \arg\min_w \; SSE(w) + \frac{\sigma^2}{\tau^2} \sum_j w_j^2 \qquad [3.6]$$

That's the ridge loss [3.2] with $\lambda = \sigma^2/\tau^2$. So **$\lambda$ is how noisy we think the data is, divided by how big we think the weights are**. Our yards-and-turnovers line misses by about 7.7 points a game ($\sigma^2 \approx 60$), so if we expected weights of about ±5 ($\tau = 5$) we'd get $\lambda \approx 2.4$. A flat prior ($\tau \to \infty$) sends λ to 0 and MAP back to MLE. A Laplace prior, $p(w_j) \propto e^{-|w_j|/b}$, gives $\lambda \Sigma |w_j|$ instead, which is LASSO.

MAP estimation

L1 and **L2 regularization** are a form of **MAP Estimation**.

MAP stands for Maximum A Posteriori which is a **Bayesian** approach for **applying a prior to our weights**.

MLE only asks one question: which weights make the data we saw the most likely? MAP asks that too, but it also asks which weights we thought were likely before we saw any data at all. That second part is the **prior**. If we believe weights are probably small and close to zero (a Gaussian prior), the math turns that belief into exactly the $\lambda \Sigma w_j^2$ term from ridge. A sharper prior that piles up right at zero (a Laplace prior) turns into the $\lambda \Sigma |w_j|$ term from LASSO instead. So **regularization is really just us telling the model what we expected ahead of time**.

Review

Complete the review: **L1 regularization** penalizes the absolute values of the weights. **L2 regularization** penalizes the squared weights. In a penalty formulation, increasing **lambda** makes the penalty stronger. In a budget formulation, increasing the budget makes the constraint looser.

Matching: regularization

E **MAP Estimation**	**A.** Penalizes the squared size of the weights, encouraging small weights
A **L2 (Ridge)**	**B.** Controls how heavily the regularization cost is weighed
D **L1 (LASSO)**	**C.** The budget the weights must remain within
B λ	**D.** Penalizes the absolute size of the weights, driving some to exactly 0
C $\mathbf{t}$	**E.** A Bayesian approach for applying a prior to our weights

Practice problems

All of these use the Notre Dame football data from these two chapters.

1. Units and the penalty. Last chapter's line is $\hat{y} = 9.0\, x_1 - 6.7$ with x_1 in hundreds of yards, or $\hat{y} = 0.090\, x_1 - 6.7$ with x_1 in plain yards. The two make exactly the same predictions. Why might ridge treat them differently?

2. Ridge or LASSO? Total yards is pass yards plus rush yards. We put all three in one model. Plain OLS gives weights of −64, +72 and +74. What would ridge do with these three features, and what would LASSO do? Which would you use to find out which stats matter?

3. Reading λ as a prior. Our yards-and-turnovers line has $\sigma^2 \approx 60$. On the MAP pages, a prior of $\tau = 5$ gave $\lambda \approx 2.4$. What λ do we get if we expect smaller weights, $\tau = 2$? Will the fitted weights get bigger or smaller?

Practice problem answers

p. 23: The penalty is on the size of the weights, not the predictions. A weight of 0.09 costs almost nothing to keep, a weight of 9.0 costs a lot more, so the same feature gets shrunk very differently depending on its units. That's why we rescale every feature to the same spread before regularizing.

p. 23: Ridge keeps all three and spreads the weight between them sensibly (with $\lambda = 1$: +5.9 on total, +2.0 on pass, +3.9 on rush). LASSO picks: on standardized features it sets pass yards to exactly 0 and keeps total yards (about 7.3 to 7.9 points per 100 yards) plus a little rush yards. LASSO is the one that tells you which stats matter, ridge is the one to use if you want to keep every feature with stable weights.

p. 23: $\lambda = \sigma^2/\tau^2 = 60/4 = 15$. Expecting smaller weights means a bigger λ, which pulls the fitted weights in closer to 0.